



DEPARTAMENTO
DE INDUSTRIAS

UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

LABORATORIO 3: AUTOMATIZACIÓN DE PROCESOS CON N8N

SISTEMAS DE INFORMACIÓN PARA LA GESTIÓN ICN292

Montserrat Rojas Araya

21497666-8, 202260583-9, paralelo 101, *montserratt.rojas@usm.cl*

02/09/2026

CAMPUS VITACURA

TABLA DE CONTENIDOS

0.1	¿Qué se hizo?	2
0.2	Principales Conclusiones	2
0.3	Evidencias de Verificación	2
1	Declaración parámetros y semilla personal	4
2	URL n8n	4
3	Arquitectura y Configuración de Flujos en n8n	4
3.1	Flujo Receptor (<i>Triage</i>)	4
3.2	Flujo Emisor (<i>Emitter</i>)	5
4	Resultados y Verificación de Ejecución Exitosa	7
5	Conclusiones	7

Resumen Ejecutivo

El presente trabajo abordó la automatización integral del proceso de recepción, evaluación y clasificación de solicitudes de devolución para un comercio electrónico mediante la plataforma de orquestación de flujos n8n. El objetivo central consistió en desacoplar la lógica operativa a través de flujos independientes: un flujo emisor para simular la carga transaccional, un flujo receptor de triage con reglas de negocio parametrizadas ($D = 21$ días y $U = \$46,000$), enriquecimiento de datos en tiempo real mediante el consumo de la API pública de mindicator.cl para la conversión a Unidades de Fomento (UF), y la consolidación de métricas de proceso.

0.1 ¿Qué se hizo?

- Flujo Triage (Receptor): Se configuró un endpoint HTTP (Webhook) en entorno productivo para recepcionar peticiones en formato JSON. Mediante un nodo Switch se estructuró un árbol de decisión para clasificar las órdenes en cuatro estados: Aprobación, Revisión, Rechazo y Datos Inválidos. Posteriormente, se enlazó una consulta externa vía HTTP Request a la API económica para obtener el valor del indicador UF del día y se consolidaron los campos de salida mediante un nodo de script (Code en JavaScript) para garantizar la integridad referencial de variables, retornando la respuesta final sincronizada al solicitante mediante Respond to Webhook.
- Flujo Emisor (Simulación): Se implementó un generador programático en Python (Code) cargado con el lote de prueba estandarizado de 15 solicitudes. La transmisión se ejecutó mediante un nodo HTTP Request bajo método POST con serialización controlada de payload (JSON.stringify).
- Flujo Resumen / Consolidación: Se estructuró el procesamiento y registro de las transacciones para su análisis global de desempeño operativo.

0.2 Principales Conclusiones

- Desacoplamiento y Escalabilidad: La separación del flujo en microservicios mediante webhooks permite procesar lotes de forma asíncrona y confiable sin bloquear los sistemas origen.
- Robustez ante Ramificaciones Condicionales: La resolución de variables dependientes de ramas no transitadas mediante lógica programática (try/catch o validación de ejecución) previene interrupciones en tiempo de ejecución (Node hasn't been executed), superando las limitaciones de nodos estáticos de asignación.
- Enriquecimiento Dinámico de Datos: La incorporación de APIs externas en línea (mindicator.cl) permite estandarizar valorizaciones en unidades reajustables (UF) con total precisión temporal (registrando valores entre 0.24 UF y 7.08 UF en el lote analizado).

0.3 Evidencias de Verificación

- Ejecuciones Exitosas: El lote completo de 15 ítems fue procesado satisfactoriamente por el flujo emisor (Workflow executed successfully), recibiendo de vuelta el 100

- Ejecuciones Fallidas Identificadas y Subsanaadas:
 - Falla por invocación a webhook de prueba inactivo (404 Webhook not registered), solucionada mediante la publicación activa (Published) del endpoint de producción.
 - Error de serialización en el cliente emisor (Unexpected token '=', -[object Object]), resuelta mediante el casteo explícito con `JSON.stringify($json)`.
 - Error de referencia de ramas condicionales en el nodo Registro y Notificación, solventado sustituyendo la expresión dependiente por código JavaScript seguro.

1 Declaración parámetros y semilla personal

En conformidad con las instrucciones del laboratorio, se declara la semilla personal S y los parámetros calculados para la parametrización de las reglas de negocio del flujo de *triage*:

- **Semilla personal (S):** 666
- **Umbral de monto (U):**

$$U = 30\,000 + 1\,000 \times (S \pmod{50}) = 30\,000 + 1\,000 \times (666 \pmod{50})$$

$$U = 30\,000 + 1\,000 \times 16 = \$46\,000$$

- **Plazo máximo (D):**

$$D = 7 + 7 \times (S$$

Resumen de parámetros aplicados en n8n:

Parámetro	Valor	Criterio de Evaluación
Semilla (S)	666	Identificador base
Umbral de monto (U)	\$46 000	Revisión si <i>monto</i> > 46 000
Plazo máximo (D)	21 días	Rechazo si <i>días</i> > 21

2 URL n8n

<https://montser55.app.n8n.cloud/webhook/177f2da9-671a-413e-856e-35879fbd8ddf>

3 Arquitectura y Configuración de Flujos en n8n

Para cumplir con el desacoplamiento de servicios y automatizar el procesamiento de devoluciones, la solución se estructuró mediante flujos de trabajo independientes dentro de la plataforma **n8n**:

3.1 Flujo Receptor (*Triage*)

El flujo de *Triage* constituye el núcleo de la lógica de negocio y opera de manera asíncrona mediante los siguientes nodos:

1. **Webhook (Endpoint HTTP):** Expuesto mediante la URL de producción bajo el método POST. Recibe la carga útil (*payload*) de la solicitud en formato JSON y delega la respuesta síncrona al nodo de cierre.
2. **Switch (Enrutador de Reglas de Negocio):** Clasifica cada registro entrante comparando sus atributos contra los parámetros personalizados ($S = 666$, $U = \$46\,000$, $D = 21$ días):
 - **DATOS_INVALIDOS:** Comprueba que los campos numéricos sean válidos y consistentes.

- **RECHAZO:** Evalúa solicitudes fuera de plazo (*dias_desde_compra* > 21) o productos categorizados como *danado_por_uso*.
 - **REVISION:** Identifica solicitudes dentro de plazo pero con montos superiores al umbral (*monto* > \$46 000) o productos con fallas técnicas (*con_fallas*).
 - **APROBACION:** Solicitudes dentro de plazo, con monto menor o igual al umbral ($\leq$ \$46 000) y producto en condición *sin_abrir* o *abierto_sin_uso*.
3. **Consulta API UF (HTTP Request):** Realiza una petición GET al servicio REST público de <https://mindicador.cl/api>, obteniendo en tiempo de ejecución el valor oficial de la Unidad de Fomento (UF) para valorizar la devolución.
 4. **Registro y Notificación (Code - JavaScript):** Script seguro que detecta qué nodo condicional fue ejecutado mediante control programático (`try/catch` sobre `isExecuted`), calculando el valor en UF (*monto_clp/uf_valor*) a dos decimales y consolidando el mensaje estandarizado para el cliente.
 5. **Respond to Webhook:** Devuelve el objeto JSON consolidado con código de estado HTTP 200 al solicitante.

3.2 Flujo Emisor (*Emitter*)

Simula el despacho por lotes de las solicitudes de prueba hacia el Webhook de producción:

1. **Trigger Manual (When clicking 'Execute workflow'):** Inicia la corrida de pruebas de manera controlada.
2. **Code in Python:** Genera y estructura en memoria el lote de 15 solicitudes de devolución en formato diccionario, entregando 15 ítems individuales a la salida del nodo.
3. **HTTP Request:** Envía secuencialmente cada uno de los 15 registros hacia el endpoint del flujo de *Triage* utilizando serialización explícita `{{ JSON.stringify($json) }}` en el cuerpo del mensaje.

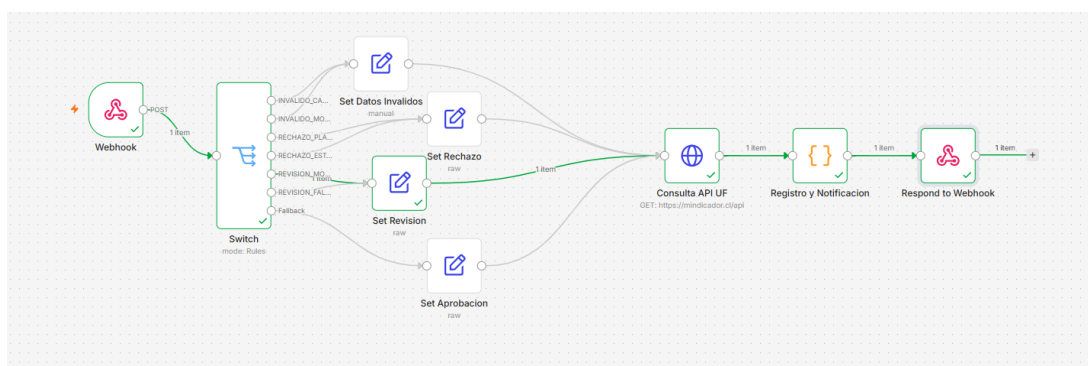
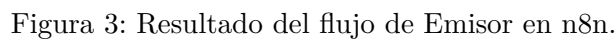


Figura 1: Lienzo general del flujo de Triage en n8n.



4 Resultados y Verificación de Ejecución Exitosa

El flujo emisor completó satisfactoriamente la transmisión y recepción del lote completo de 15 solicitudes contra el flujo de *Triage*. La Tabla 1 presenta la respuesta devuelta por el sistema:

Cuadro 1: Resultados consolidados de la ejecución de las 15 solicitudes de prueba.

ID Solicitud	SKU	Monto (CLP)	Monto (UF)	Ruta	Motivo
DEV-1115	AU-210	\$19 990	0.49	APROBACION	Cumple política directa
DEV-1116	TE-330	\$69 980	1.71	REVISION	Monto superior a umbral ($> \$46\,000$)
DEV-1117	HV-050	\$24 990	0.61	REVISION	Producto con fallas
DEV-1118	MW-150	\$79 990	1.95	REVISION	Monto superior a umbral ($> \$46\,000$)
DEV-1119	PL-120	\$45 990	1.12	RECHAZO	Fuera de plazo ($25 > 21$ días)
DEV-1120	AS-260	\$89 990	2.20	REVISION	Monto alto y producto con fallas
DEV-1121	LV-330	\$289 990	7.08	REVISION	Monto superior a umbral ($> \$46\,000$)
DEV-1122	PL-120	\$45 990	1.12	APROBACION	Cumple política directa
DEV-1123	AU-210	\$39 980	0.98	RECHAZO	Producto dañado por uso
DEV-1124	HV-050	\$49 980	1.22	REVISION	Monto superior a umbral ($> \$46\,000$)
DEV-1125	TE-330	\$34 990	0.85	RECHAZO	Fuera de plazo ($22 > 21$ días)
DEV-1126	MW-150	\$79 990	1.95	REVISION	Monto superior a umbral y fallas
DEV-C01	CB-010	\$9 990	0.24	APROBACION	Caso de control: parámetros base
DEV-C02	LV-330	\$289 990	7.08	REVISION	Caso de control: monto crítico
DEV-C03	MW-150	\$79 990	1.95	RECHAZO	Caso de control: plazo excedido ($29 > 21$ días)

5 Conclusiones

La experiencia práctica demostró la efectividad de las arquitecturas basadas en orquestación ligera para la automatización de procesos operacionales. Las principales lecciones aprendidas abarcan:

- La importancia de la persistencia de endpoints productivos frente a entornos de prueba al interconectar flujos asíncronos.
- La necesidad de utilizar programación defensiva mediante scripts al evaluar flujos con ramificaciones lógicas disjuntas.
- La viabilidad de enriquecer flujos transaccionales en tiempo real mediante el consumo directo de microservicios REST externos sin degradación en el rendimiento.